

Particles collision simulation with generative neural networks

Marcello Pozzessere

December 3, 2022

Abstract

In this paper we explored some simple methods of creating and training neural networks with the goal of reproducing (fictional) particles collisions. Two incoming electrons with fixed energy 200 GeV collide, and a Monte Carlo simulation created the four-momenta of two outgoing hypothetical particles with an arbitrary (equal) mass in the range 10-150 GeV. Using this data, firstly, a regressional neural network was trained to learn the mass of the particles from energy and momenta. Secondly, a variational autoencoder was studied to create a generative network able to mimic the original data.

It was found that even simple networks with 2-3 layers of 100s of nodes were able to estimate the mass and its uncertainty to an acceptable level. The VAE was, on the other hand, less performing and the produced data that did not reproduce the original distributions. Furthermore, th

Introduction on generative machine learning

In many physics applications simulations are necessary for experiments and data analysis. But, simulations can be complicated and computationally expensive, so working around this

problem would allow to more easily produce data in large quantities.

The way this could be achieved by generative neural networks. Simply put, a NN can be trained, using already generated data, to learn how to reproduce a simulation.

Even though the training can be long and we need enough data to begin with, this method has the great advantage that utilizing a network to produce new data is instead computationally cheap and fast.

Many architectures can be used for this purpose, as Variational Autoencoder (VAE) and Generative Adversarial Networks (GAN).

1 Monte Carlo simulation

In this simple setup an electron e^- and a positron e^+ with equal energy 200 GeV (and neglectable mass) collide producing two hypothetical identical particles x_1 and x_2 .

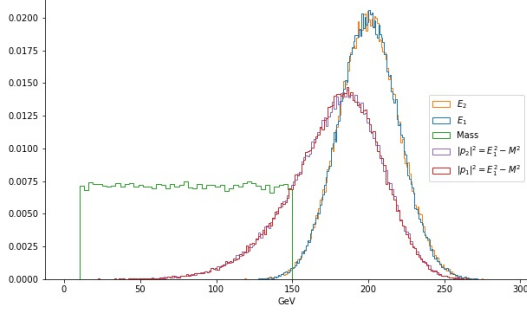
The mass m_X of the unknown particle was sampled from a uniform distribution between 10 GeV and 150 GeV. Given the energy-momentum:

$$p_i^\mu = p_{e^-}^\mu + p_{e^+}^\mu = (2E_{e^-}, 0, 0, 0)$$

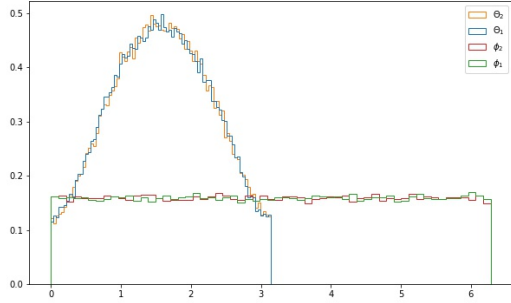
$$p_f^\mu = p_{x_1}^\mu + p_{x_2}^\mu$$

$$p_f^\mu = (2E_{x_1}, p_{x_1} + p_{x_2}, p_{y_1} + p_{y_2}, p_{z_1} + p_{z_2})$$

the four-momentum conservation: $p_i^\mu = p_f^\mu$ and



(a) Energy, mass and momenta distribution (normalized to 1)



(b) Angles θ and ϕ distribution (normalized to 1)

the relativistic dispersion relation: $E_{x_1}^2 - \mathbf{p}_{x_1}^2 = m_X^2$, it can be found:

$$E_x = E_e = 200 \text{ GeV}$$

$$p_{i_1} = p_{i_2} \quad (i = x, y, z)$$

$$(\mathbf{p}_1)^2 = (\mathbf{p}_2)^2 = E_e^2 - m_x^2$$

In this simplified scheme it was also assumed that every possible direction of the outgoing particles was equally probable i.e. uniformly distributed on a spherical surface. Mathematically $P(\Omega)d\Omega = \frac{1}{4\pi}d\Omega$ implies that θ should not be uniformly distributed (in 4 the full explanation).

The distributions of energy, mass and angles are shown in figure 1a and 1b.

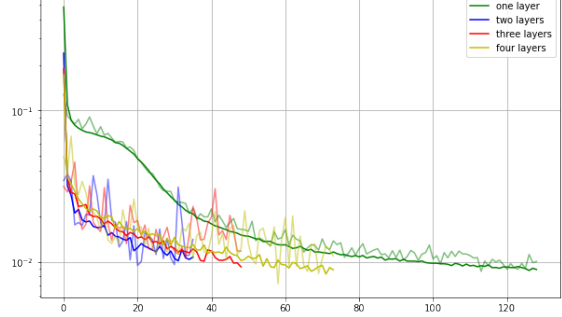


Figure 2: Loss and Validation loss

2 Regressional Neural network

Firstly, a regressional NN can be trained to return the mass of the particles. Supervised training was used: the inputs were two four-momenta and the label was the mass, with the loss function being the "mean squared error". To visualize the accuracy of each architecture the true mass was plotted against the estimated mass.

2.1 Optimization

To improve the network some simple hyperparameter optimization was performed. The performances were overall very similar with deeper networks converging faster (for the same batch size).

Parameters	Values tried	Final
# of layers	1, 2, 3, 4	3
size of layers	100, 200, 300, 400	[100,200,100]
activation	relu, leakyrelu, elu	relu

Additionally, different initialization of the weights were explored but they did not lead to significant changes, so the default `glorot_uniform` was used.

2.2 Regularization

To prevent overfitting, "early stopping" was used. The loss function was computed on the validation dataset and when this value started increasing (due to overfitting) the training was stopped.

2.3 Uncertainty

To estimate the uncertainties of the mass, two methods were used and compared:

From the network

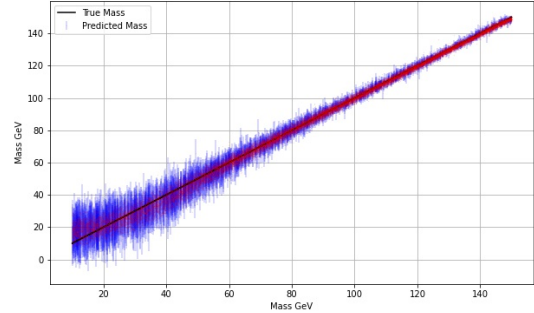
The network was modified to output two values: μ_M and σ_M . The loss function was also adjusted to:

$$LOSS = \frac{1}{N} \sum_i \left(2 \log \sigma + \left(\frac{y_{true} - \mu}{\sigma} \right)^2 \right)$$

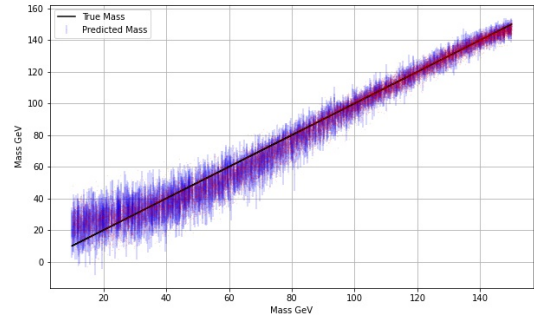
The result is shown in figure 3a.

From MC dropout

An alternative way is the Monte Carlo dropout: a network with layers that, with probability $p = 10\%$, will be "turned off" during each learning iteration. The predicted mass was obtained by averaging 100 of these networks, keeping the dropout active and thus finding a distribution of masses for each input. From this distribution, mean and standard deviation were extracted.



(a) Predicted mass over true mass, with uncertainty estimated from the network



(b) Predicted mass over true mass, with uncertainty estimated from MC dropout

3 Vae

3.1 Structure

A normal autoencoder consists of two parts: an encoder that tries to condense the input into some latent space, and a decoder that tries to

recreate the input starting from the same latent space. What a variational autoencoder does is to force this latent space to be a known distribution, usually a (multivariate) gaussian distribution.

This method was used to train a network (the decoder) able to produce new data. The loss function used was a sum of "mse" and "kl-divergence":

$$LOSS_{mse} = \frac{1}{N} \sum_i (y_{true} - \hat{y})^2$$

$$LOSS_{kl} = \frac{1}{2} \sum_i (1 + \log(Z_{\sigma_i}^2) - Z_{\mu_i}^2 - Z_{\sigma_i}^2)$$

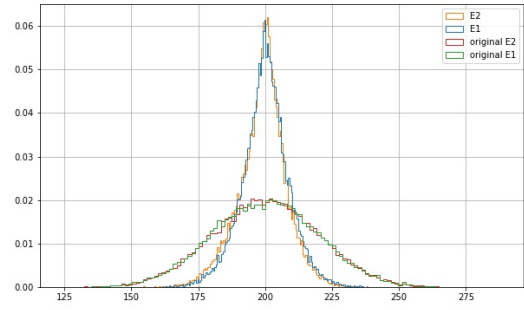
Z_{μ} and Z_{σ} are the parameters of the gaussian distribution of the latent space.

3.2 Results

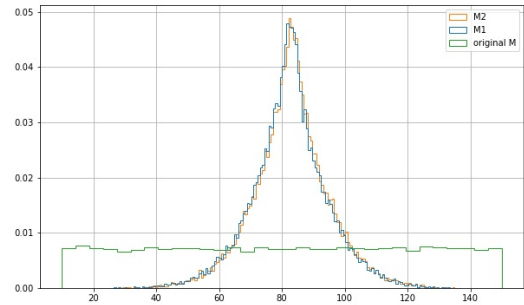
Unfortunately the results of this network were not satisfactory. Even by adjusting different hyperparameters the distribution of the created events did not match the original very well, as shown in one example in figure 4a and 4b.

4 Conclusions

In conclusion, while the regressional networks were quite successful, the generative model seemed to be still inadequate. Furthermore, being able to input a mass in the generative network would make it more useful in producing simulations data but that would require the latent space to match exactly the "mass space" and this remained a challenge.



(a) Predicted energy distribution from VAE



(b) Predicted mass distribution from VAE

Appendix

Uniform distribution on a spherical surface

To have a uniform distribution [1] : $P(\Omega) = \frac{1}{4\pi}$

$$P(\Omega)d\Omega = P(\theta, \phi)d\theta d\phi$$

$$d\Omega = \sin \theta d\theta d\phi$$

$$P(\theta, \phi) = \frac{\sin \theta}{4\pi}$$

We can then integrate to find the distributions of the single variables:

$$P(\phi) = \int_0^{2\pi} P(\theta, \phi)d\theta = \frac{\sin \theta}{2}$$

$$P(\theta) = \int_0^\pi P(\theta, \phi)d\phi = \frac{1}{2\pi}$$

To generate random variables with these distributions we can invert the cumulative density functions (CDF) $F(\phi)$ and $F(\theta)$

$$F(\theta) = \int_0^\theta P(\theta')d\theta' = \int_0^\theta \frac{\sin \theta'}{2}d\theta' = \frac{1 - \cos \theta}{2}$$

$$F(\phi) = \int_0^\phi P(\phi')d\phi' = \int_0^\theta \frac{1}{2\pi}d\phi' = \frac{\phi}{2\pi}$$

and finally:

$$\theta = F^{-1}(v) = \cos^{-1}(2v - 1)$$

$$\phi = F^{-1}(u) = 2\pi u$$

where u and v are random variables uniformly distributed between 0 and 1

References

1. [https : / / www . bogotobogo . com / Algorithms / uniform _ distribution _ sphere.php](https://www.bogotobogo.com/Algorithms/uniform_distribution_sphere.php).